核心源码

第一步：先看懂源代码

```python
import dill
import numpy as np
import pandas as pd
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder

# 数据加载和预处理部分保持不变
columsNames = [
    'duration', 'protocol_type', 'service', 'flag',
'src_bytes', 'dst_bytes',
    'land', 'wrong_fragment', 'urgent', 'hot',
'num_failed_logins', 'logged_in',
    'num_compromised', 'root_shell', 'su_attempted',
'num_root', 'num_file_creations',
    'num_shells', 'num_access_files', 'num_outbound_cmds',
'is_host_login',
    'is_guest_login', 'count', 'srv_count', 'serror_rate',
'srv_serror_rate',
    'rerror_rate', 'srv_rerror_rate', 'same_srv_rate',
'diff_srv_rate',
    'srv_diff_host_rate', 'dst_host_count',
'dst_host_srv_count',
    'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
'dst_host_same_src_port_rate',
    'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',
'dst_host_srv_serror_rate',
    'dst_host_rerror_rate', 'dst_host_srv_rerror_rate',
'class', 'difficulty'
]

dataTrain = pd.read_csv(r"KDDTrain+.txt", header = None,
```

```
names = columsNames)
dataTest = pd.read_csv(r"KDDTest+.txt", header = None, names
= columsNames)

dataTrain = dataTrain.drop('difficulty', axis=1)
dataTest = dataTest.drop('difficulty', axis=1)

attack_mapping = {
    # DOS攻击
    'back': 'DOS', 'land': 'DOS', 'neptune': 'DOS', 'pod':
'DOS',
    'smurf': 'DOS', 'teardrop': 'DOS', 'apache2': 'DOS',
'udpstorm': 'DOS',
    'processtable': 'DOS', 'worm': 'DOS', 'mailbomb': 'DOS',
# mailbomb只存在于测试集中，相当于零日攻击
    # PROBE攻击
    'satan': 'PROBE', 'ipsweep': 'PROBE', 'nmap': 'PROBE',
'portsweep': 'PROBE',
    'mscan': 'PROBE', 'saint': 'PROBE',

    # R2L攻击
    'guess_passwd': 'R2L', 'ftp_write': 'R2L', 'imap':
'R2L', 'phf': 'R2L',
    'multihop': 'R2L', 'warezmaster': 'R2L', 'warezclient':
'R2L', 'spy': 'R2L',
    'xlock': 'R2L', 'xsnoop': 'R2L', 'snmpguess': 'R2L',
'snmpgetattack': 'R2L',
    'httptunnel': 'R2L', 'sendmail': 'R2L', 'named': 'R2L',

    # U2R攻击
    'buffer_overflow': 'U2R', 'loadmodule': 'U2R',
'rootkit': 'U2R', 'perl': 'U2R',
    'sqlattack': 'U2R', 'xterm': 'U2R', 'ps': 'U2R',

    # 正常流量
    'normal': 'NORMAL'
}
```

```python
# 将class列映射到5分类
dataTrain['class'] = dataTrain['class'].map(attack_mapping)
dataTest['class'] = dataTest['class'].map(attack_mapping)

categorical_columns = ['protocol_type', 'service', 'flag']

# 先对离散特征进行factorize编码（在划分x_train和x_test之前）
for col in categorical_columns:
    dataTrain[col], unique_categories =
pd.factorize(dataTrain[col]) # 对训练集进行factorize
    # pd.factorize用于将字符串变成数字ID
    category_mapping = {cat: idx for idx, cat in
enumerate(unique_categories)} # 创建映射字典
    dataTest[col] = dataTest[col].map(category_mapping) # 对
测试集使用相同的编码（遇到新类别会赋值为-1）
    dataTest[col] = dataTest[col].fillna(-1).astype(int) #
将测试集中未见的类别设置为-1

# 现在再划分特征和目标变量
x_train = dataTrain.drop('class', axis=1)
y_train = dataTrain['class']
x_test = dataTest.drop('class', axis=1)
y_test = dataTest['class']

# 对数值特征进行标准化（对每一列特征进行）
numerical_columns = [col for col in x_train.columns if col
not in categorical_columns]
scaler = StandardScaler()
# 初始化一个标准化缩放器 将数值特征转换为均值为0 标准差为1的正态
分布 让不同特征训练时属于同级
x_train[numerical_columns] =
scaler.fit_transform(x_train[numerical_columns])
x_test[numerical_columns] =
scaler.transform(x_test[numerical_columns])
# 训练集的数值特征拟合+转换 测试集的只做转换避免泄露数据

# 创建指示函数
y_train_binary = np.where(y_train == 'NORMAL', 0, 1)
```

```
y_test_binary = np.where(y_test == 'NORMAL', 0, 1)

# 2. 计算各类别的平均特征向量（使用训练集计算）
class_labels = ['NORMAL', 'DOS', 'PROBE', 'U2R', 'R2L']
class_mean_vectors = {}

for class_label in class_labels:
    # 获取当前类别的所有样本
    class_mask = (y_train == class_label)
    if np.sum(class_mask) > 0:
        # 计算该类别所有样本的特征平均值
        class_data = x_train[class_mask]
        mean_vector = class_data.mean(axis=0).values  # 转换
为numpy数组
        class_mean_vectors[class_label] = mean_vector
        print(f"类别 {class_label} 的平均特征向量形状:
{mean_vector.shape}, 样本数量: {np.sum(class_mask)}")
    else:
        print(f"警告: 训练集中没有找到类别 {class_label} 的样
本")
        class_mean_vectors[class_label] =
np.zeros(x_train.shape[1])
# 生成了布尔掩码 获取属于当前类别的样本 用掩码提取类别特征子矩阵
并计算均值

def create_label_vectors(labels, mean_vectors_dict):
    """为标签列表创建对应的平均特征向量表示"""
    label_vectors = []
    for label in labels:
        label_vectors.append(mean_vectors_dict[label])
    return np.array(label_vectors)

# 为训练集和测试集创建标签向量
# 把y变成矩阵 行: 样本数 列: 特征数
y_train_vectors = create_label_vectors(y_train,
class_mean_vectors)
y_test_vectors = create_label_vectors(y_test,
class_mean_vectors)
```

```python
print("\n各类别平均特征向量示例 (前5个特征):")
for class_label, mean_vector in class_mean_vectors.items():
    print(f"{class_label}: {mean_vector[:5]}")

# 转换为PyTorch张量
x_train_tensor = torch.FloatTensor(x_train.values)  # 训练数据转tensor
y_train_vectors_tensor = torch.FloatTensor(y_train_vectors)  # 标签向量转tensor
y_train_binary_tensor = torch.FloatTensor(y_train_binary)  # 二分类标签转tensor
x_test_tensor = torch.FloatTensor(x_test.values)
y_test_vectors_tensor = torch.FloatTensor(y_test_vectors)
y_test_binary_tensor = torch.FloatTensor(y_test_binary)
# 目前有三样东西:
# 1.x: 原始流量 喂给左边的encoder
# 2.y_vector:理想特征 喂给右边的encoder
# 3.y_binary:算loss判断对没对

# 创建数据加载器
batch_size = 256  # 可以根据你的硬件调整批次大小
train_dataset = TensorDataset(x_train_tensor, y_train_vectors_tensor, y_train_binary_tensor)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
# shuffle=true:打乱顺序 防止模型偷懒
test_dataset = TensorDataset(x_test_tensor, y_test_vectors_tensor, y_test_binary_tensor)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

# 保存预处理数据
filename_save = 'NSL_KDD_Load.pkl'
dill.dump_session(filename_save)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```python
print(f"使用设备: {device}")

# 加载预处理数据
filename_save = 'NSL_KDD_Load.pkl'
dill.load_session(filename_save)

# Bidirectional Anchor Contrastive Loss
class BACLoss(nn.Module):
    def __init__(self, device, temperature=0.1):
        # 温度系数这么低 模型就可以放大相似度的差异 让模型对相似和不相似更加敏感
        super(BACLoss, self).__init__()
        self.device = device
        self.temperature = temperature

    def forward(self, logits, labels):
        # logits: 一个n*n的矩阵 logits[i][j]表示第i个样本和第j个标签向量之间的相似度
        logits /= self.temperature  # 放大相似度 放大差异
        logits_normalX = logits[(labels == 0).squeeze()]  # 只把属于‘NORMAL'(0)的样本选出来 筛选行
        logits_normalX_normalY = logits_normalX[:, (labels == 0).squeeze()]  # 在刚刚筛选的行里面找列也是’NORMAL‘的部分 作分子
        logits_normalX_abnormalY = logits_normalX[:, (labels == 1).squeeze()]  # 找筛选的行里面列是’ATTACK‘的部分 是分母的一部分
        denominatorX = torch.exp(logits_normalX_normalY) + torch.sum(torch.exp(logits_normalX_abnormalY), axis=1, keepdims=True)
        # 计算分母 利用softmax+交叉熵损失的公式
        log_probs_X = logits_normalX_normalY - torch.log(denominatorX)
        loss_X = -log_probs_X
        loss_X = loss_X.mean()
        # 这一段以lossx 的视角出发 以样本为视角的损失 作为正常样本要找正常原型
```

```
        logits_normalY = logits[:, (labels == 0).squeeze()]
# 这一次先筛选列 只看正常标签列
        logits_normalY_normalX = logits_normalY[(labels ==
0).squeeze()]  # 筛选行也为正常的样本
        logits_normalY_abnormalX = logits_normalY[(labels ==
1).squeeze()]  # 行是攻击样本 不应该和正常原型相似
        denominatorY = torch.exp(logits_normalY_normalX) +
torch.sum(torch.exp(logits_normalY_abnormalX), axis=0,
keepdims=True)
        # 依旧计算分母 但是axis=0 沿着列的方向求和
        log_probs_Y = logits_normalY_normalX -
torch.log(denominatorY)
        loss_Y = -log_probs_Y
        loss_Y = loss_Y.mean()
        # 以lossy为视角 作为正常原型需要尽可能多的覆盖召回所有的
正常样本 并且排斥攻击样本

        loss = loss_X + loss_Y
        return loss
# BACLoss是一个基于对比学习的损失函数 专门用于单分类或异常检测情
况 最大化正常样本和正常原型的相似度 最小化正常样本与异常原型的相
似度

class Encoder(nn.Module):
    def __init__(self, input_dim):
        super(Encoder, self).__init__()
        nearest_power_of_2 = 2 **
round(math.log2(input_dim))
        second_layer_size = nearest_power_of_2 // 2
        third_layer_size = nearest_power_of_2 // 4
        # input_dim:输入的特征向量 先把输入维度调整到最近的2的
幂次 然后每次减半计算中间层维度
        # 自适应能力强 不管什么数据集 都会自动设计出一个倒金字塔
的结构
        self.network = nn.Sequential(
            # 第一层
            nn.Linear(input_dim, second_layer_size),  # 线性
层（全连接）
```

```
            nn.BatchNorm1d(second_layer_size),  # 批归一化
            nn.ReLU(),  # 激活函数 特征加入非线性 让网络能够学会复杂的规律
            nn.Dropout(0.2),  # 随机丢弃 防止过拟合
            # 第二层
            nn.Linear(second_layer_size, third_layer_size),
            nn.BatchNorm1d(third_layer_size),
            nn.ReLU(),
            # 第三层（输出层）
            nn.Linear(third_layer_size, 32)
        )

    def forward(self, x):
        return self.network(x)

# 这个类实现一个双塔结构
class ContrastiveModel(nn.Module):
    def __init__(self, feature_dim, label_dim):
        super(ContrastiveModel, self).__init__()
        self.feature_encoder = Encoder(feature_dim)
        # 引擎1：负责处理真实的流量特征x
        self.label_encoder = Encoder(label_dim)
        # 引擎2：负责处理抽象的标签向量y
    def forward(self, features, labels):
        # 编码
        z_x = self.feature_encoder(features)  # 把流量压缩成32维
        z_y = self.label_encoder(labels)  # 把标签压缩成32维
        # 归一化 便于计算余弦相似度
        z_x = F.normalize(z_x, p=2, dim=1)
        z_y = F.normalize(z_y, p=2, dim=1)
        # 计算相似度
        similarity_matrix = torch.matmul(z_x, z_y.T)
        return similarity_matrix

# 获取特征和标签的维度
feature_dim = x_train.shape[1]
label_dim = y_train_vectors.shape[1]
```

```python
# 模型初始化
model = ContrastiveModel(feature_dim, label_dim).to(device)
criterion = BACLoss(device=device, temperature=0.1)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# 分批次训练函数
def train_model(model, train_loader, criterion, optimizer,
device, epochs=50):
    model.train()
    # 每一个epoch大周期内 模型都把百分百的数据看了一遍
    for epoch in range(epochs):
        total_loss = 0.0
        batch_count = 0
        # batch小周期
        for batch_idx, (features, label_vectors,
binary_labels) in enumerate(train_loader):
            features = features.to(device)
            label_vectors = label_vectors.to(device)
            binary_labels = binary_labels.to(device)
            optimizer.zero_grad()  # 梯度清零
            similarity_matrix = model(features,
label_vectors)  # 前向传播 模型算出相似度矩阵
            loss = criterion(similarity_matrix,
binary_labels)
            loss.backward()  # 反向传播 通过链式法则算出怎么改
每一个参数使loss变小
            optimizer.step()  # 参数更新
            total_loss += loss.item()
            batch_count += 1
        avg_loss = total_loss / batch_count
        print(f'Epoch {epoch+1}/{epochs} completed. Average
Loss: {avg_loss:.6f}')

    return model

# 评估函数
def evaluate_model(model, test_loader, device):
```

```
    model.eval()
    total_loss = 0.0
    batch_count = 0
    # 关闭梯度计算
    with torch.no_grad():
        for features, label_vectors, binary_labels in
test_loader:
            # 搬运数据
            features = features.to(device)
            label_vectors = label_vectors.to(device)
            binary_labels = binary_labels.to(device)
            # 前向传播
            similarity_matrix = model(features,
label_vectors)
            # 计算损失
            loss = criterion(similarity_matrix,
binary_labels)
            total_loss += loss.item()
            batch_count += 1
    avg_loss = total_loss / batch_count
    print(f'Test Loss: {avg_loss:.6f}')
    return avg_loss

# 开始训练
print("开始分批次训练...")
trained_model = train_model(model, train_loader, criterion,
optimizer, device, epochs=50)

# 评估模型
print("开始评估模型...")
test_loss = evaluate_model(trained_model, test_loader,
device)
```

核心步骤：

- 统计每种攻击的平均特征向量
- 双塔映射同时处理标签和真实流量并压缩到32维 拉到一个公共的空间
- 利用BACLoss把正常流量拉进正常平均原型 推开远离异常平均原型

- 最终测试 看流量与哪一类的平均特征向量余弦相似度最大 就归为哪一类

第二步：创新点分析

创新点一：标签的语义化增强

- 传统做法：
  - 标签是正交的 没有物理意义 模型不知道DOS和Probe有什么本质区别
- 本文做法：
  - 计算了每一类攻击在训练集上的统计均值
  - label encoder接受的不是0和1 而是这一类攻击的平均特征向量

创新点二：双向锚点对比损失 （BACLoss）

- 传统做法：无监督 通过数据增强制造正样本对
- 本文做法：
  - 监督对比
  - 双向约束： 样本找原型 原型找样本
  - 不对称关注： 拼命把正常流量压缩的很紧 建立严格的白名单

创新点三：从分类到度量的范式转移

- 传统的**NIDS**：`Feature -> Model -> Probability`
- 本文的**NIDS**：`Feature -> Encoder -> Feature Space <- Encoder <- Prototype` 泛化性强

2.5

分析效率低的原因：
在特征筛选这一步 做法过于冗余
为了解决特征冗余问题 必须由简入繁地迭代 所以每次评估一个特征 都需要重新训练一遍神经网络 时间复杂度：$O(N * TrainTime)$

过于保守的原因： 算法最后留下来太多的特征 剔除不够狠 是因为**Encoder**的拟合能力太强了 导致即使特征很烂 Encoder也可以让loss降下去 导致$dertaLoss$ 很小 算法认为该特征不重要 或者相反 因为随机噪声也没能干扰到模型 所以阈值太低 什么特征都舍不得抛弃

优化算法的可能性?

1. 之前的办法每删除一个特征就重新训练模型 那不如不要重练

- 输入层后面加一个权重层 每一个特征对应一个权重

- 在loss里面加一个正则项 强迫大部分w_i 变成0
- 结果： 只需要训练一次 看谁的权重是0 谁就是没用的特征（思路借鉴来源：FSNID）
- 效果：复杂度由$O(N)$ 变成$O(1)$

2:针对BACLoss的优化

- 如果正常样本对都是差别不大的 导致梯度消失 模型最后躺平
- 那么算loss的时候就可以不要平均算 专门挑最难分 的样本算loss 加大权重 强迫模型不能偷懒 必须分清楚模棱两可的边界样本

第三步：个人意见（？）

逻辑漏洞?

- 预处理阶段计算了每一类攻击的均值向量 但是网络攻击流量模态多样化 用一个均值代表一类复杂的攻击 不会导致欠拟合吗

数据泄漏/静态缺陷?

- 测试阶段依旧使用的是训练集计算出来的 `class_mean_vectors` 但是如果测试集出现了严重的概念漂移 模型就会变得没有适应性

第四点：个人思考改进可能性

可学习的原型：

- 之前的均值是算死的 不可动
- 改进思考：把y_vector的生成过程去掉 在网络里随机初始化一个矩阵 让其参与梯度下降 这样模型就可以在训练过程中摸索出来最完美的5个中心点 （从FSNID借鉴来的想法）

引入对抗训练：

- 之前的模型只学习了简单的样本
- 改进思考：训练时给流量特征x增加轻微扰动 强迫模型面对对抗样本也能匹配到正确的原型 可以增强模型的鲁棒性